

Goal

In this online exam, you must add an IP to the Aquila SoC to compute the inner product of two 1×8 **integer** vectors. In short, you want to compute $c = A \cdot B$, where each entry of the vectors A and B is a **signed** 8-bit integer and c is a **signed** 32-bit integer that stores the inner product value. The IP has 16 8-bit registers to store A and B , a 32-bit registers to store c , and a 32-bit control/status register ' $trig$ '. These registers can be accessed by the processor through the memory-mapped I/O (MMIO) interface. The ELF file of a program (see Figure 1) will be given to you to test the IP. The program is compiled for TCM so your Aquila do not have to enable DRAM and FPU.

Exam Regulations

- [1] You can download code or the Power Point slides from E3 to the test computer between 10:10 am ~ 10:25 am. Any downloading behavior outside this period will be regarded as cheating.
- [2] You are not allowed to use your mobile phone, browse the internet for any references, and talk to any other classmates. If you have any questions, please raise your hand and the TA will talk to you.
- [3] You are allowed **2 hours** for this online exam.
- [4] After your circuit is completed and verified, please upload all your **modified** Verilog files to E3. You have to leave the computer classroom immediately afterwards.

Exam Descriptions

Your IP module, `dotprod`, should use the address `0xC2000000` as the base address of the register array that stores vector $A[]$, the base address `0xC2000010` for $B[]$, and the base address `0xC2000020` for c . In addition to the register array for $A[]$, $B[]$, and c , we have one more 32-bit register `trig` used to trigger the activation of the `dotprod` IP and return the status of the computed result. **The register `trig` and c should be set to zero upon system reset**, but it is not necessary to reset the registers $A[]$ and $B[]$. The declaration and function of these registers should be as follows:

1. `reg signed [7:0] A[0:7];`
 $A[]$ is an 8-element signed 8-bit array that stores the 1×8 vector A . This array is readable/writable from the CPU though the MMIO interface.
2. `reg signed [7:0] B[0:7];`
 $B[]$ is an 8-element signed 8-bit array that stores the 1×8 vector B . This array is readable/writable from the CPU though the MMIO interface.
3. `reg signed [31:0] c;`
 c is a signed 32-bit register that stores the result of inner product of $A \cdot B$. This array is read-only from the CPU though the MMIO interface. Any write operations from the CPU to c shall be ignored.
4. `reg [31:0] trig;` is a 32-bit unsigned register that when its value is set to non-zero by the CPU, the IP `dotprod` will begin to compute inner product of A and B . As soon as the valid result is stored in c , the IP should set `trig` to 0 to inform the CPU that the calculation is done. This register is readable/writable from the CPU.

Grading policy:

1. [30%] Correct read/write operations of A and B register arrays.
2. [40%] Correct computations of inner product and the read-only c register. Note that you can **only use one multiplier** (i.e. DSP Slice) in your `dotprod` IP. Some points will be deducted if you violate this rule and uses more than one multiplier.
3. [30%] Correct operation of the control/status register `trig`.

Final comment:

You cannot modify the software. You must use the ELF file we gave you to test your program using full-system simulation. The ELF file is compiled from the program you see in Figure 1.

```

#include <stdio.h>
#include <string.h>

char volatile    *A = (char *) 0xC2000000;
char volatile    *B = (char *) 0xC2000010;
int  volatile    *c = (int  *) 0xC2000020;
int  volatile    *trig = (int  *) 0xC2000030;

char a_init[] = { 17, -24, 33, -6, 88, -43, 12, 0 };
char b_init[] = {-127, -4, 9, 3, 73, 23, 31, 41 };

int main(void)
{
    int i;

    // Copy the A[] and B[] vectors into the IP
    memcpy((void *) A, (void *) a_init, sizeof(a_init));
    memcpy((void *) B, (void *) b_init, sizeof(b_init));

    // Print the vectors A[] and B[]
    printf("A[]={");for(i=0;i<8;i++)printf("%5d,",A[i]);printf(" }\n");
    printf("B[]={");for(i=0;i<8;i++)printf("%5d,",B[i]);printf(" }\n\n");

    // Triger the IP to find mix/max values
    *trig = 1;
    while (*trig) /* busy loop */; // Waiting for IP to respond

    printf("The inner product value by the IP is: %d.\n", *c);
    printf("The result %s correct!\n\n", (*c == 4023)? "is" : "isn't");

    return 0;
}

```

Figure 1. The test program of the ELF file.

Note: TAs will use different program to verify your circuit after the exam so you shall not hardcode the solution into to your IP.